

MedVault AI — Authentication, Doctor Verification & Emergency Access Module

I am developing a healthcare platform called **MedVault AI**.

MedVault AI is a centralized cloud-based digital healthcare record management platform that allows patients to securely store, organize, and access their lifelong medical records. The platform also provides AI-powered multilingual medical document summarization and a secure emergency-access mechanism for verified medical practitioners.

The system has three primary roles:

- **Patient**
- **Doctor / Medical Practitioner**
- **Admin**

Build the authentication, registration, doctor-verification, and emergency-access modules according to the following requirements.

1. Patient Registration

Create a dedicated patient signup flow.

Required fields

- Full Name
- Date of Birth
- Gender
- Mobile Number
- Email Address
- Password
- Confirm Password
- Address
- City
- State
- PIN Code
- Emergency Contact Name

- Emergency Contact Number
- Relationship with Emergency Contact

Optional medical profile fields

These should NOT be mandatory during account creation:

- Blood Group
- Known Allergies
- Existing Medical Conditions
- Current Medications

These can be completed later from the patient's Medical Profile.

Identity

Each patient should automatically receive a unique:

MedVault ID

Example:

MV-10293847

Government identity integration such as Aadhaar may be supported in the future, but it should NOT be mandatory for the prototype.

Do not store unnecessary Aadhaar information.

2. Doctor Registration

Doctor registration must be completely separate from patient registration.

A doctor must NOT receive access to patient records simply by creating an account.

Doctor registration should contain two stages:

Stage 1 — Account Creation

Collect:

- Full Name
- Date of Birth
- Mobile Number
- Email Address

- Password
- Confirm Password
- Profile Photo
- Address
- City
- State
- Hospital / Clinic Name
- Hospital / Clinic Address

Stage 2 — Professional Verification

Collect:

- Medical Registration Number
- State Medical Council
- Registration Date
- Medical Qualification
- Medical College / Institution
- Graduation Year
- Specialization
- Hospital / Clinic
- Medical Registration Certificate
- Degree Certificate
- Professional Identification Document, if applicable

The most important verification information is:

Medical Registration Number + State Medical Council

3. Doctor Verification System

The platform must NOT consider a doctor verified merely because they uploaded a degree certificate.

The main question is:

Is this person a currently registered and authorized medical practitioner?

Implement a dedicated:

Doctor Verification Service

The verification flow should be:

Doctor Signup

- Professional Information
- Medical Registration Number
- State Medical Council
- Verification Service
- Official Medical Registry / Authorized Registry
- Match Doctor Information
- Verification Result

The system should attempt to verify:

- Doctor Name
- Medical Registration Number
- State Medical Council
- Registration Status
- Registration Date
- Professional Qualification

If an official registry/API integration is available, use it for automated verification.

If an official API is NOT available for the prototype, implement a **manual verification workflow** where an administrator verifies the submitted registration details against the appropriate official medical register.

DO NOT claim that an automated NMC/API verification exists unless an actual authorized API integration has been implemented.

4. Doctor Verification Status

Every doctor account must have a verification status.

Possible statuses:

- PENDING
- VERIFIED

- REJECTED
- SUSPENDED
- EXPIRED

Default status after registration:

PENDING

A doctor with PENDING, REJECTED, or SUSPENDED status must NOT be allowed to access patient medical records.

Only a VERIFIED doctor can request patient-record access.

Display a clear verification badge in the doctor dashboard:

Verified

 Verified Medical Practitioner

or

Pending

 Verification Pending

or

Rejected

 Verification Rejected

5. Admin Verification Dashboard

Create an Admin dashboard for doctor verification.

The administrator should be able to see:

- Doctor Name
- Registration Number
- State Medical Council
- Qualification
- Specialization
- Hospital
- Submitted Documents

- Verification Status
- Registration Date
- Submission Date

Admin actions:

- Approve
- Reject
- Request Additional Information
- Suspend Doctor
- View Verification Documents

When approving a doctor, store:

- Verified By
- Verification Date
- Verification Source
- Verification Notes

6. Doctor Verification Principle

Do not rely exclusively on uploaded certificates.

Use this verification hierarchy:

1. Medical Registration Number
2. State Medical Council
3. Registration Status
4. Identity Match
5. Professional Qualification
6. Supporting Documents

A degree certificate is supporting evidence.

The medical registration status is the primary verification criterion.

7. Emergency Access System

One of the major features of MedVault AI is:

Verified Emergency Doctor Access

A verified doctor should be able to access critical patient information during an emergency when the patient is unconscious, unresponsive, or unable to communicate.

However, verification alone should NOT provide unrestricted access to every patient's complete medical history.

Implement controlled emergency access.

Emergency Access Flow

Doctor Login

- Verify Doctor Account
- Search / Identify Patient
- Request Emergency Access
- Enter Reason
- System Validates Doctor
- Grant Appropriate Emergency Access
- Record Access in Audit Log

Example emergency reason:

Patient unconscious and unable to provide medical history.

8. Emergency Medical Information

Emergency access should initially expose only critical information such as:

- Patient Name
- MedVault ID
- Blood Group
- Known Allergies
- Critical Medical Conditions
- Current Medications
- Major Previous Surgeries
- Important Medical History
- Recent Prescriptions
- Critical Diagnostic Information

- Emergency Contact

Provide an option for expanded medical-record access only when properly authorized.

9. Emergency Access Reason

Every emergency-access request must require a reason.

Example reasons:

- Patient unconscious
- Patient unable to communicate
- Emergency treatment
- Unknown medical history
- Ambulance / emergency care
- Other

If "Other" is selected, require a written explanation.

10. Audit Logging

Every access to patient medical records must be logged.

Create an access_logs system.

Store:

- Log ID
- Doctor ID
- Patient ID
- Timestamp
- Access Type
- Reason
- IP Address
- Records Accessed
- Action Performed
- Authorization Status

Example:

Doctor:

Dr. Rahul Sharma

Patient:

MV-102938

Access Type:

Emergency

Reason:

Patient unconscious

Records Accessed:

- Allergies
- Medications
- Blood Group
- Previous Surgery

Timestamp:

11-Aug-2026 19:42

Status:

Authorized

Patients should be able to view their access history so they know which verified practitioners accessed their records.

11. Database Structure

Create the following logical database entities.

users

Fields:

- user_id
- email
- mobile
- password_hash
- role

- account_status
- created_at
- updated_at

Roles:

- PATIENT
- DOCTOR
- ADMIN

patients

Fields:

- patient_id
- user_id
- medvault_id
- full_name
- date_of_birth
- gender
- address
- city
- state
- pin_code
- blood_group
- emergency_contact_name
- emergency_contact_number
- emergency_contact_relationship
- created_at
- updated_at

doctors

Fields:

- doctor_id
- user_id
- full_name
- date_of_birth
- medical_registration_number
- state_medical_council
- registration_date
- qualification
- medical_college
- graduation_year
- specialization
- hospital_name
- hospital_address
- verification_status
- verification_source
- verified_by
- verified_at
- verification_notes
- created_at
- updated_at

doctor_documents

Fields:

- document_id
- doctor_id
- document_type
- document_url

- uploaded_at
- verification_status
- verified_by
- verified_at

Document types:

- MEDICAL_REGISTRATION_CERTIFICATE
 - DEGREE_CERTIFICATE
 - PROFESSIONAL_ID
 - OTHER
-

access_logs

Fields:

- log_id
 - doctor_id
 - patient_id
 - access_type
 - reason
 - records_accessed
 - ip_address
 - timestamp
 - authorization_status
-

12. Role-Based Access Control

Implement strict RBAC.

Patient

Patient can:

- Create account
- Login

- View profile
- Upload medical records
- View medical records
- Update medical profile
- Manage emergency information
- View AI-generated summaries
- Select preferred language
- View doctor access history
- Manage pediatric profiles
- Control permissions where applicable

Patient CANNOT:

- Verify doctors
- Access another patient's records
- Approve doctor accounts

Doctor

Doctor can:

- Create account
- Submit professional credentials
- View verification status
- Login after account creation
- Access their own profile
- Search patients according to authorized workflows
- Request emergency access
- View authorized patient records
- View emergency medical information
- View relevant medical documents

Doctor CANNOT:

- Access patient records while unverified
 - Modify patient records without authorization
 - Approve themselves
 - Approve other doctors
-

Admin

Admin can:

- Manage users
- Review doctor applications
- Verify doctors
- Reject doctor applications
- Suspend doctors
- View submitted credentials
- Manage verification status
- Review audit logs
- Handle verification issues

Admin should NOT be able to casually modify patient medical information unless a specific administrative workflow requires it.

13. Pediatric Profiles

MedVault AI must support children and newborns.

Parents or legal guardians should be able to create a pediatric profile.

Pediatric profile should contain:

- Child Name
- Date of Birth
- Gender
- Birth Information
- Blood Group

- Vaccination Records
- Medical History
- Allergies
- Medications
- Parent / Guardian Information
- Emergency Contact

The child profile remains under parental/guardian control while the child is a minor.

Design the system so that the profile can transition to an independent patient account when the child reaches the configured legal age.

Do not hard-code legal assumptions; make the age threshold configurable.

14. Security Requirements

MedVault AI is a healthcare application, so security must be treated as a core requirement.

Implement:

- HTTPS/TLS
- Password hashing using a strong password-hashing algorithm
- JWT or secure session-based authentication
- Role-Based Access Control
- AES-256 encryption for sensitive stored data where appropriate
- Secure cloud storage for medical documents
- File type validation
- File size validation
- Malware/security scanning where available
- Audit logging
- Rate limiting
- Input validation
- SQL injection protection
- XSS protection

- CSRF protection where applicable
 - Secure API authentication
 - Secure document URLs
 - No sensitive information in frontend logs
 - No passwords stored in plaintext
 - No medical documents stored in publicly accessible buckets
-

15. Authentication Flow

Implement:

Patient

Signup

→ Email/Mobile Verification

→ Login

→ Patient Dashboard

Doctor

Signup

→ Email/Mobile Verification

→ Professional Credential Submission

→ PENDING

→ Verification

→ VERIFIED

→ Doctor Dashboard

→ Patient Access Features Enabled

Admin

Login

→ Admin Dashboard

→ Doctor Verification

16. Doctor Dashboard

The doctor dashboard should clearly show:

Profile

- Doctor Name

- Qualification
- Specialization
- Registration Number
- State Medical Council
- Hospital
- Verification Status

Patient Access

- Search Patient
- Emergency Access
- Authorized Records
- Access History

Verification

If pending:

Your medical credentials are currently under verification. Patient-record access will be enabled after successful verification.

If verified:

- Your medical practitioner account has been verified.

17. Patient Dashboard

The patient dashboard should contain:

- MedVault ID
- Personal Profile
- Medical Records
- Upload Document
- AI Medical Summary
- Emergency Profile
- Medications
- Allergies

- Vaccinations
 - Pediatric Profiles
 - Doctor Access History
 - Privacy & Security Settings
-

18. Important Product Principle

MedVault AI should follow this principle:

Authentication proves who the user is. Authorization determines what the user can access. Verification establishes whether a doctor is a legitimate medical practitioner. Audit logging records what happened.

Do not treat these as the same thing.

19. Future Medical Registry Integration

Design the Doctor Verification Service as a separate module so that an official medical-registry integration can be added later.

Use an abstraction such as:

DoctorVerificationService

Possible implementations:

OfficialRegistryVerificationService

ManualAdminVerificationService

This allows the prototype to work with manual verification while keeping the architecture ready for an authorized registry/API integration in the future.

Do not scrape or automate access to government websites in a way that violates their terms or bypasses access controls.

20. Final Objective

The final system should demonstrate that:

1. Anyone can create a patient account.
2. A person claiming to be a doctor cannot immediately access patient records.

3. Doctors must provide professional registration information.
4. Doctor credentials are verified before patient access is enabled.
5. Unverified doctors cannot access medical records.
6. Verified doctors can request emergency access.
7. Emergency access is limited to appropriate critical information.
8. Every access is recorded in an immutable/auditable log.
9. Patients can see who accessed their records.
10. The architecture is ready for integration with an authorized medical registry in the future.
11. Pediatric profiles are supported.
12. Patient medical information is protected using strong security and access-control mechanisms.

Build the UI and backend around these requirements. Prioritize security, privacy, clear role separation, and realistic healthcare workflows over unnecessary UI complexity.